

INTELIGENCIA ARTIFICIAL (1INF24)



UNIDAD 1: Introducción a la IA. Búsqueda y optimización en IA

**Tema 3: Algoritmos heurísticos, metaheurísticos y bioinspirados
(Parte 2)**

MSc. Luis Muroya

Adaptado de material: Dr. Edwin Villanueva

Contenido

- Inteligencia Colectiva
- Algoritmos de Inteligencia Colectiva
 - Particle Swarm Optimization (PSO)
 - Artificial Bee Colony (ABC)



- ¿Qué es la inteligencia?
 - > Difícil de definir, involucra capacidad de aprender de la experiencia, recordar conocimientos, inferir, resolver problemas.
- ¿Es una característica netamente humana?
 - > No. Diferentes especies manifiestan diversas formas de inteligencia.
- ¿Dónde reside la inteligencia?
 - El cerebro
- ¿Existe inteligencia sin cerebro?
 - > Sí

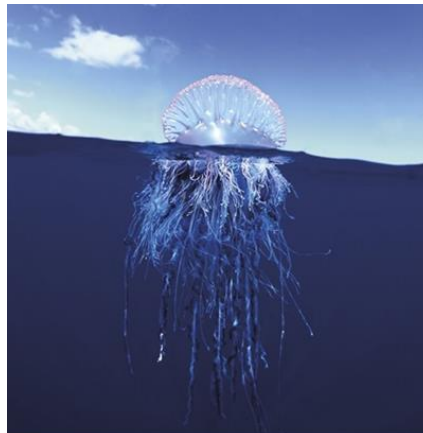
Inteligencia Colectiva (CI)

- No tienen cerebro, pero sí tienen células nerviosas especializadas llamadas ganglios.
- Otros incluso carecen de estas células, pero tienen células nerviosas entrelazadas en el cuerpo, red neuronal.



Pepinos de mar

*Alta capacidad de
mímica al entorno.*



Carabelas portuguesas

*Colonia de zooides que
viven juntos desde
nacimiento. Cada uno
tiene un rol.*



Medusas

*Colores brillantes que
advienten peligro.*

Inteligencia Colectiva (CI)

- Se define CI como el aprendizaje que se efectúa a nivel de grupo. La base de este tipo de inteligencia es la experiencia compartida.
- CI (o Swarm intelligence)¹ fue introducida por Gerardo Beni, Suzanne Hackwood y Jing Wang en 1989, cuando investigaban las propiedades de agentes auto-organizados para **sistemas robóticos celulares***. Se inspiraron en el comportamiento social de colonias de insectos, como las hormigas.



Las habilidades de estos sistemas sociales parecen trascender las habilidades individuales.

Capacidades y fortalezas grupales



(*) Sistema compuesto por unidades autónomas, auto-organizadas y robustas (células)

Se deben a un pequeño grupo de simples interacciones de bajo-nivel entre individuos y su entorno

Ejemplo: Particle Swarm Optimization (PSO)

Conjunto de técnicas inspiradas en el comportamiento social de bandadas de aves, bancos de peces y otros.



Bandadas de Aves



Cardúmenes de Peces

Ejemplo: *Artificial Bee Colony (ABC)*

Conjunto de técnicas inspiradas por las actividades de una colonia de abejas.



Panales de Abejas

Ejemplo: *Ant Colony Optimization (ACO)*

Conjunto de técnicas inspiradas por las actividades de una colonia de hormigas



Colonias de Hormigas

Propiedades Generales:

- Algoritmos **estocásticos**, basados en **poblaciones** y usan representación en **punto flotante**.
 - **Basado en poblaciones:** trabaja con un conjunto de soluciones de forma simultánea, pudiendo existir interacciones entre ellas.
 - **Representación en punto flotante:** cada individuo se representa como un número real y no una cadena binaria.
- Fueron diseñados intentando solucionar problemas de optimización desafiantes.
- Estos algoritmos controlan pocos parámetros (fáciles de usar)
- Tienen buenas propiedades de convergencia.



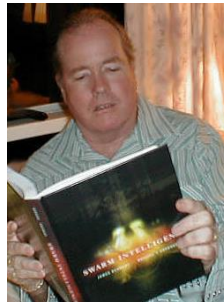
Particle Swarm Optimization (PSO)



Particle Swarm Optimization (PSO)

- En 1995, PSO fue desarrollado por **James Kennedy** y **Russell Eberhart**², para resolver problemas de optimización continua

James
Kennedy



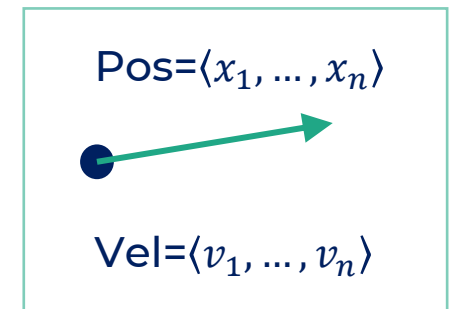
Russell
Eberhart

- Su trabajo se inspiró en el comportamiento y dinámica de los movimientos de los pájaros, insectos y peces. Usaron las reglas de **cohesión** y **alineamiento** del modelo de Craig Reynolds.
- PSO, al igual que AG es un método basado en poblaciones, pero diferentemente de los AG, la metáfora por detrás es la **cooperación** en lugar de la rivalidad. En PSO, los miembros del enjambre nunca mueren.

[2] Kennedy, J.; Eberhart, R. (1995). Particle Swarm Optimization. Proceedings of IEEE International Conference on Neural Networks Vol. IV: 1942–1948.

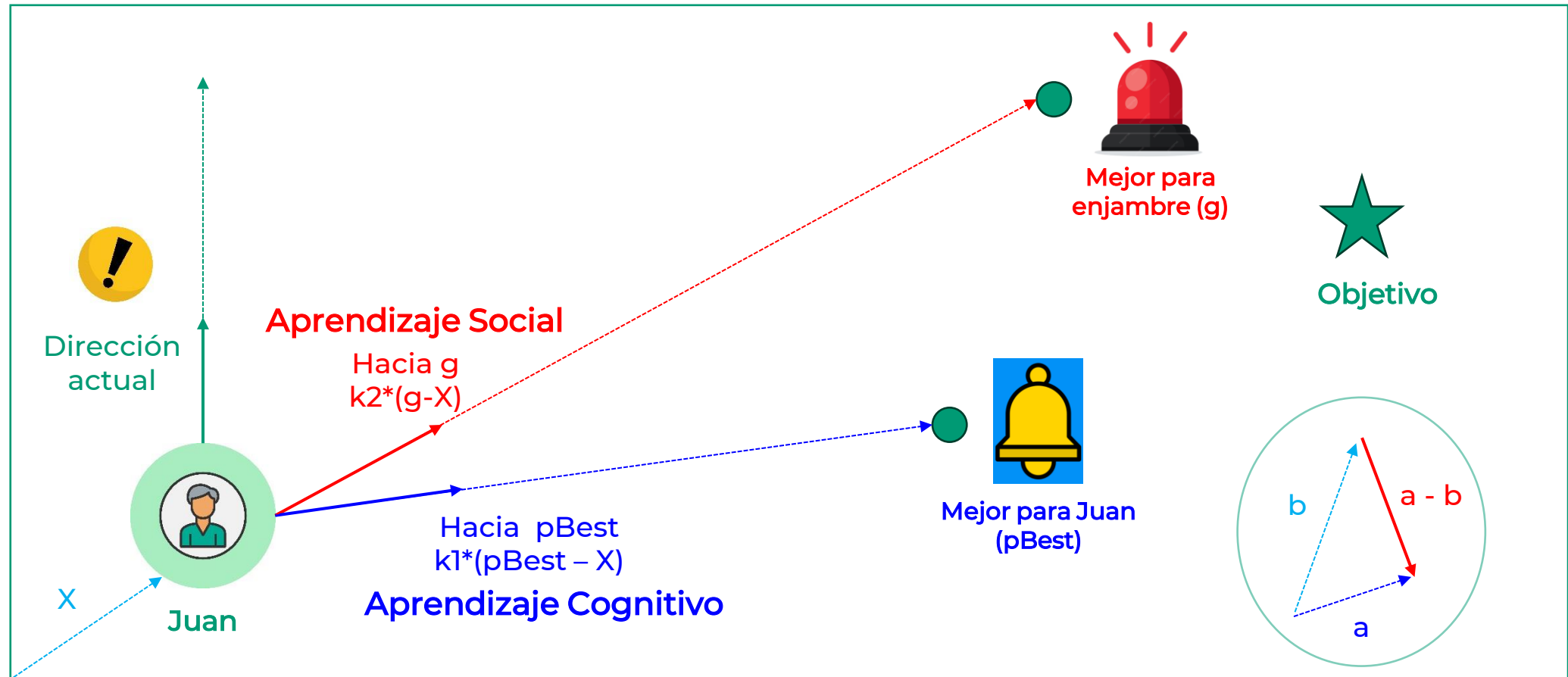
Particle Swarm Optimization (PSO)

- Una población en PSO se compone de un **conjunto de partículas** que representan las soluciones a un problema.
- En cada iteración, todas las partículas se **mueven por el espacio** del problema con el objetivo de encontrar una solución óptima global. Cada partícula busca, almacena y comunica su mejor solución
- Una partícula en PSO tiene:
 - Un vector de su **posición** actual.
 - Un vector de **velocidad** que dirige su movimiento.
 - La capacidad de intercambiar información con sus vecinos.
 - La capacidad de memorizar una posición anterior.



Particle Swarm Optimization (PSO)

Intuición (Día 1, 10km diarios)



Particle Swarm Optimization (PSO)

Intuición (Día 2, 10km diarios)



Particle Swarm Optimization (PSO)

Modelo matemático:

Una partícula **i** en la interacción $k+1$ se mueve a una posición x_i^{k+1} que depende de su posición anterior (x_i^k) y su nueva velocidad (v_i^{k+1}):

$$x_i^{k+1} = x_i^k + v_i^{k+1} \quad (1)$$

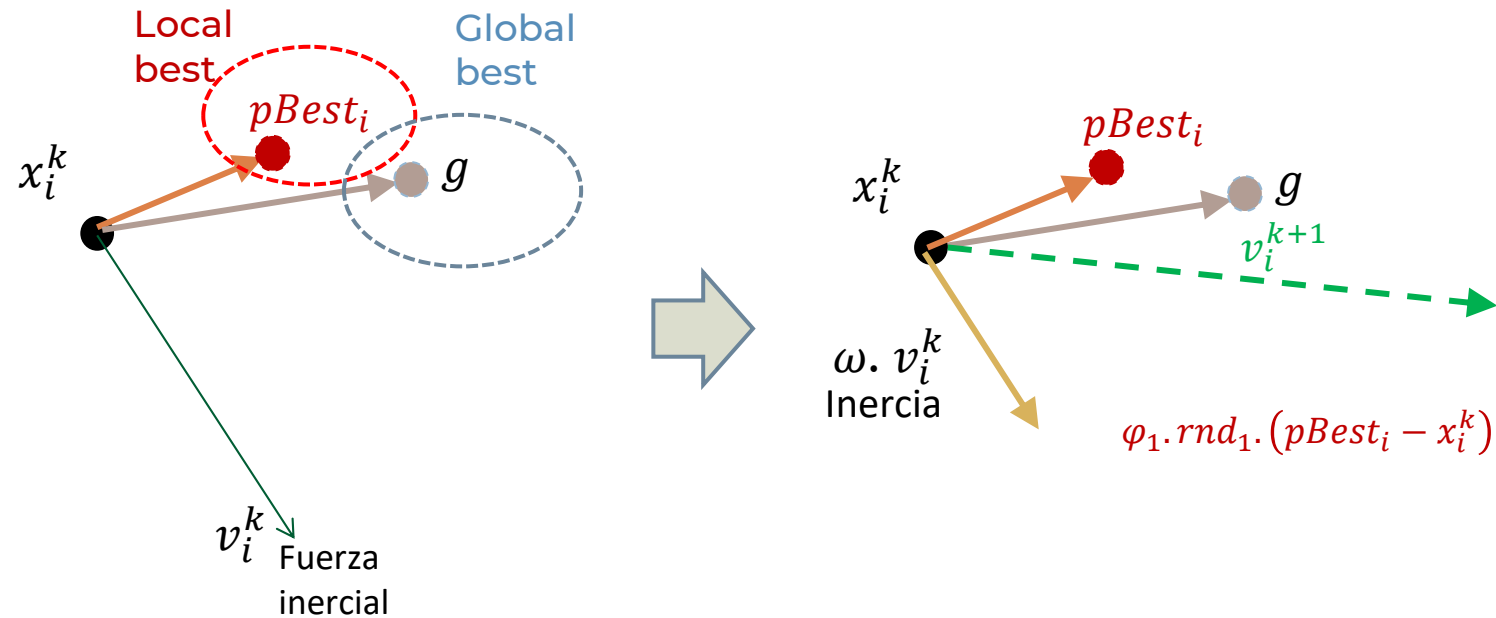
$$v_i^{k+1} = \underbrace{\omega \cdot v_i^k}_{\text{Fuerza inercial}} + \underbrace{\varphi_1 \cdot \text{rnd}_1 \cdot (pBest_i - x_i^k)}_{\text{Aprendizaje Cognitivo: Fuerza hacia la mejor posición local } pBest_i} + \underbrace{\varphi_2 \cdot \text{rnd}_2 \cdot (g - x_i^k)}_{\text{Aprendizaje Social: Fuerza hacia la mejor posición global } g \text{ del enjambre.}} \quad (2)$$

Donde:

- v_i^k , velocidad anterior
- ω representa el factor inercia
- $pBest_i$, mejor posición encontrada por partícula i -ésima
- g , mejor posición global ya vista por el conglomerado
- φ_1 y φ_2 , pesos de tendencia individual o social respectivamente.
- rnd_1 y rnd_2 , son números aleatorios entre $\{0,1\}$.

Particle Swarm Optimization (PSO)

Movimiento vectorial de partículas:



$$v_i^{k+1} = \omega \cdot v_i^k + \varphi_1 \cdot \text{rnd}_1 \cdot (pBest_i - x_i^k) + \varphi_2 \cdot \text{rnd}_2 \cdot (g - x_i^k)$$

$$x_i^{k+1} = x_i^k + v_i^{k+1}$$

Aprendizaje
Cognitivo

Aprendizaje Social

Particle Swarm Optimization (PSO)

Implementación (Pseudocódigo):

Algorithm 2 Pseudocodigo del algoritmo PSO Canónico

```
1:  $S \leftarrow \text{SwarmInitialization}()$ 
2: while not stop condition do
3:   for each particle  $x_i$  of the swarm  $S$  do
4:     evaluate( $x_i$ )
5:     if  $\text{fitness}(x_i)$  is better than  $\text{fitness}(pBest_i)$  then
6:        $pBest_i \leftarrow x_i$ 
7:     end if
8:     if  $\text{fitness}(pBest_i)$  is better than  $\text{fitness}(g)$  then
9:        $g \leftarrow pBest_i$ 
10:    end if
11:  end for
12:  for each particle  $x_i$  of the swarm  $S$  do
13:     $v_i^{k+1} = \omega \cdot v_i^k + \varphi_1 \cdot rnd_1 \cdot (pBest_i - x_i^k) + \varphi_2 \cdot rnd_2 \cdot (g - x_i^k)$ 
14:     $x_i^{k+1} = x_i^k + v_i^{k+1}$ 
15:  end for
16: end while
17: Output: best solution found
```

Particle Swarm Optimization (PSO)

Elección de parámetros:

$$v_i^{k+1} = \omega \cdot v_i^k + \varphi_1 \cdot \text{rnd}_1 \cdot (pBest_i - x_i^k) + \varphi_2 \cdot \text{rnd}_2 \cdot (g - x_i^k)$$

- Algunas recomendaciones de valores para:
 - Inercia de la partícula a $\omega = 0.5$, porque debe influir menos.
 - Para φ_1 y $\varphi_2 = 2$ (tendencia local φ_1 y tendencia global del enjambre φ_2).
- Una mejora del método consiste en empezar con una inercia relativamente alta ($\omega = 1.4$) que se vaya reduciendo en cada iteración, por ejemplo, multiplicándola por un factor $r < 1$ para estabilizar el sistema.
- Otras propuestas:

$$\text{Inertia weight } \omega = (\omega_1 - \omega_2) \left(\frac{\text{maxiter} - \text{iter}}{\text{iter}} \right) + \omega_2,$$

$$\text{Inertia weight } \omega_i = \left(1.1 - \frac{gbest_i}{(pbest_i)} \right).$$

Hacer linealmente decaer en el tiempo la inercia



Particle Swarm Optimization (PSO)

Consideraciones:

- **Población inicial:** crea un conjunto S de partículas, donde a cada partícula x_i se le asigna una posición inicial y una velocidad, ambas deben estar acotadas por límites:

$$x_i \leftarrow \cup (inf, sup) \quad v_i \leftarrow \cup (-|sup - inf|, |sup - inf|)$$

- **Iteraciones:**
 - Tras actualizar todas las partículas, se comparan sus mejores posiciones $pBest$ con la mejor posición global g , si algún $pBest$ es mejor que g , entonces: $g \leftarrow pBest_i$.
 - Al acabar las iteraciones, g contiene la mejor solución encontrada por el algoritmo

Particle Swarm Optimization (PSO)

Consideraciones:

- Los algoritmos tipo PSO vienen siendo aplicados en problemas de optimización combinatoria, donde la dimensión del espacio completo de soluciones es exponencial (NP-hard) con respecto a la dimensión de la representación del problema.
- A pesar de que PSO inicialmente se formuló para trabajar con representación real, están surgiendo nuevas versiones que permiten trabajar con representaciones discretas.
 - Un ejemplo es la versión de PSO basada en espacios geométricos (*Geometric PSO*)³. Las partículas se influyen dependiendo de su distancia en el espacio. El algoritmo se modifica para trabajar con soluciones binarias utilizando la distancia de Hamming (cantidad de bits diferentes entre dos cadenas).

[3] A. Moraglio, C. Di Chio, and R. Poli. Geometric Particle Swarm Optimization. In 10th European conference on Genetic Programming (EuroGP2007), volume 4445 of Lecture Notes in Computer Science. Springer, Abril 2007.

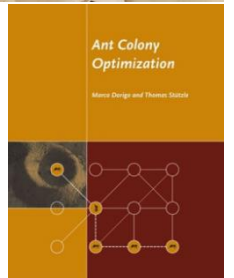
Artificial Bee Colony (ABC)



© CanStockPhoto.com - csp48357116

Artificial Bee Colony (ABC)

- El algoritmo de colonias de abejas o *Artificial Bee Colony* (ABC) ⁶, fue presentado por **Dervis Karaboga** en 2005.
- ABC es inspirado en el comportamiento inteligente observado en las abejas domesticas.
- ABC es un algoritmo propuesto para resolver problemas de optimización. Está basado en poblaciones en el cual los individuos, llamados fuentes de comida, son modificados por las abejas artificiales.
 - El objetivo de estas abejas es descubrir los lugares de comida con néctar, hasta finalmente encontrar el lugar con mayor cantidad de néctar (optimo global).



[6] D. Karaboga. An idea based on honey bee swarm for numerical optimization. Tr06, Erciyes University, Engineering Faculty, Computer Engineering Department, 2005.

Artificial Bee Colony (ABC)

- En un sistema ABC, las abejas artificiales se mueven por el espacio de búsqueda eligiendo fuentes de néctar dependiendo de su **experiencia pasada** y de su **compañera de colmena**.



Algunas abejas se mueven aleatoriamente sin influencia externa (exploradoras). Cuando encuentran una fuente de néctar mayor, memorizan su posición y olvidan la anterior.

- De este modo, ABC combina métodos de búsqueda local y búsqueda global, intentando equilibrar el balance entre **exploración** y **explotación** del algoritmo.

Artificial Bee Colony (ABC)

Modelo Biológico:

El modelo biológico que guía la búsqueda de néctar es un proceso de optimización y consta de los siguientes elementos:

- 1) **Fuente de alimento:** Es un valor numérico que indica su potencial rentabilidad.
- 2) **Abejas empleadas (recolectoras):** Exploran una fuente de alimento. Comunican su ubicación y rentabilidad, con una cierta probabilidad, a sus demás compañeras observadoras.
- 3) **Abejas desempleadas:** Están en constante búsqueda de una fuente de alimento. Hay dos tipos:

Exploradoras (Scout): se encargan de buscar en el entorno que rodea a la colmena nuevas fuentes de alimento.

Observadoras (onlooker bee): permanecen en la colmena para elegir alguna de las fuentes de alimento.



Artificial Bee Colony (ABC)

Algoritmo Básico:

0. Población inicial: es iniciada con un número **SN** de vectores de valores reales n-dimensionales, generados de la siguiente manera:

$$x_i = x_{min} + rand(0,1) \cdot (x_{max} - x_{min}) \quad (1)$$

1. Fase de búsqueda por las abejas empleadas (employed bee phase):

- **Creación de nueva fuente:** las abejas empleadas representan operadores de variación. Cuando visitan fuentes de alimento, calculan una nueva posición $v_{i,g}$, así:

$$v_{i,g} = x_i + rand(-1,1) * (x_i - x_k) \quad (2)$$

donde:

- x_i , fuente de alimento donde se encuentra la abeja en ese momento.
- x_k , fuente de alimento seleccionada aleatoriamente y diferente de x_i .
- g es el ciclo actual

Artificial Bee Colony (ABC)

Algoritmo Básico:

1. Fase de búsqueda por las abejas empleadas (employed bee phase):

- **Cálculo de aptitud:** a cada fuente de alimento se calcula el resultado de la función objetivo, f . A partir de f , se calcula la aptitud (fitness): si $f \geq 0$, se aplica $\text{fit} = 1/(1+f)$; caso contrario, se aplica $\text{fit} = 1 + \text{abs}(f)$.
- **Greedy-selection:** una vez que se obtiene $v_{i,g}$, se evaluará y se comparará con x_i , luego se selecciona la mejor dependiendo de los valores de aptitud (*fitness*) que representan. Si la aptitud de $v_{i,g}$ es mejor que la de x_i , entonces $v_{i,g}$ reemplazará x_i , es decir $x_i \leftarrow v_{i,g}$, y se convertirá en un nuevo miembro de la población; de lo contrario x_i se conserva.



Artificial Bee Colony (ABC)

Algoritmo Básico (continuación):

1. Fase de búsqueda por las abejas empleadas (employed bee phase):

- **Cuenta de intentos (trials):** cada fuente de alimento viene acompañada de un contador inicializado en 0. Si en la etapa de selección voraz (greedy-selection) la aptitud de $v_{i,g}$ es mejor que la de x_i , entonces $trial_i \leftarrow 0$; de lo contrario $trial_i$ se incrementa en una unidad.

Artificial Bee Colony (ABC)

Ejemplo Employed Bee Phase:

$$f = x + y + z$$

Fuente de Alimento

$x_i \rightarrow x_1$	6 9 8
x_2	1 2 3
$x_k \rightarrow x_3$	9 8 7
x_4	3 1 4
x_5	3 2 7

3 10 9

$$\begin{cases} v_i = [6 \ 9 \ 8] + 1 * ([6 \ 9 \ 8] - [9 \ 8 \ 7]) \\ v_i = [6 \ 9 \ 8] + [-3 \ 1 \ 1] = [3 \ 10 \ 9] \end{cases}$$

Artificial Bee Colony (ABC)

Ejemplo Employed Bee Phase:

$$f = x+y+z$$

	Fuente de Alimento	Objetivo	Fitness	Trial
$x_i \rightarrow x_1$	6 9 8	23	$1/24 = 0.040$	0
$v_i \rightarrow v_1$	3 10 9	22	$1/23 = 0.043$	
x_2	1 2 3			
x_3	9 8 7			
x_4	3 1 4			
x_5	3 2 7			



Artificial Bee Colony (ABC)

Ejemplo Employed Bee Phase:

$$f = x + y + z$$

	Fuente de Alimento	Objetivo	Fitness	Trial
x_1	3 10 9	22	$1/23 = 0.043$	0
$x_i \rightarrow x_2$	1 2 3	3 2 7	$\begin{cases} v_i = [1 \ 2 \ 3] - 1 * ([1 \ 2 \ 3] - [3 \ 2 \ 7]) \\ v_i = [1 \ 2 \ 3] - [-2 \ 0 \ -4] = [3 \ 2 \ 7] \end{cases}$	
x_3	9 8 7			
x_4	3 1 4			
$x_k \rightarrow x_5$	3 2 7			



Artificial Bee Colony (ABC)

Ejemplo Employed Bee Phase:

$$f = x+y+z$$

	Fuente de Alimento	Objetivo	Fitness	Trial
x_1	3 10 9	22	$1/23 = 0.043$	0
$x_i \rightarrow x_2$	1 2 3	6	$1/7 = 0.14$	0
$v_i \rightarrow v_2$	3 2 7	12	$1/13 = 0.08$	
x_3	9 8 7			
x_4	3 1 4			
x_5	3 2 7			



Artificial Bee Colony (ABC)

Ejemplo Employed Bee Phase:

$$f = x + y + z$$

	Fuente de Alimento	Objetivo	Fitness	Trial
x_1	3 10 9	22	$1/23 = 0.043$	0
x_2	1 2 3	6	$1/7 = 0.14$	1
$x_i \rightarrow x_3$	9 8 7			
x_4	3 1 4			
x_5	3 2 7			

Repetir
↓



Artificial Bee Colony (ABC)

Algoritmo Básico (continuación):

2. Fase de selección por las abejas observadoras (onlooker bee):

- Las abejas observadoras **seleccionan** las fuentes de alimento de acuerdo a una probabilidad p_i asociada a la fuente de alimento y este valor es proporcional a la cantidad de néctar que tiene la solución. Se calcula de la siguiente manera:

$$p_i = \frac{fit_i}{\sum_{n=1}^{SN} fit_n} \quad (3)$$

donde:

- fit_i es el valor de la aptitud (fitness) de la solución i ,
- SN es el número de fuentes de alimento..



2. Fase de selección por las abejas observadoras (onlooker bee):

- ## Tarea: investigar cada uno.

- Después de seleccionar la fuente de alimento, las abejas observadoras crearán una nueva posición candidata en el vecindario de la fuente de alimento seleccionado utilizando (2). Cada abeja observadora debe procesar una fuente de alimento.



Artificial Bee Colony (ABC)

Ejemplo Onlooker Bee Phase (muestreo aceptación-rechazo cíclico):

Resultado de la fase Employed Bee

	Fuente de Alimento	Fitness	Trial	Probabilidad
x_1	6 9 8	$1/23 = 0.043$	0	$0.043/0.438=0.09$
x_2	1 2 3	$1/7=0.14$	1	$0.14/0.438=0.32$
x_3	6 8 1	$1/16=0.06$	0	$0.06/0.438=0.14$
x_4	3 1 3	$1/8=0.125$	0	$0.125/0.438=0.29$
x_5	3 2 7	$1/13=0.07$	1	$0.07/0.438=0.16$
		$S=0.438$		

Artificial Bee Colony (ABC)

Ejemplo Onlooker Bee Phase (muestreo aceptación-rechazo cíclico):

$$f = x+y+z$$



$r=0.2$



Onlooker Bees

Fuente de Alimento

Prob

Fitness Trial

x_1

6 9 8

0.09

0.043

0

x_2

1 2 3

0.32

0.14

1

x_3

6 8 1

0.14

0.06

0

x_4

3 1 3

0.29

0.125

0

x_5

3 2 7

0.16

0.07

1

Artificial Bee Colony (ABC)

Ejemplo Onlooker Bee Phase (muestreo aceptación-rechazo cíclico):

$$f = x + y + z$$



Onlooker Bees

$r=0.10$

Fuente de Alimento

Prob

Fit

Trial

x_1

6 9 8

0.09

0.043

0

x_2

1 2 3

0.32

0.14

1

x_3

6 8 1

0.14

0.06

0

x_4

3 1 3

0.29

0.125

0

x_5

3 2 7

0.16

0.07

1

Nueva fuente*

2 2 4

Fit**

0.11

* La nueva fuente se calcula igual que en la fase 1 (Employed Bee Phase): escogiendo una fuente al azar y aplicando la ecuación (2).

** Se aplica una selección greedy usando el fitness (fit) como criterio. A mayor fit, mejor fuente.



Artificial Bee Colony (ABC)

Ejemplo Onlooker Bee Phase (muestreo aceptación-rechazo cíclico):

$$f = x+y+z$$



Onlooker Bees

$r=0.90$

	Fuente de Alimento	Prob	Fit	Trial
x_1	6 9 8	0.09	0.043	0
x_2	1 2 3	0.32	0.14	2
x_3	6 8 1	0.14	0.06	0
x_4	3 1 3	0.29	0.125	0
x_5	3 2 7	0.16	0.07	1

Artificial Bee Colony (ABC)

Ejemplo Onlooker Bee Phase (muestreo aceptación-rechazo cíclico):

$$f = x + y + z$$



Onlooker Bees

$r=0.80$

Fuente de Alimento

Prob

Fit

Trial

x_1

6 9 8

0.09

0.043

0

x_2

1 2 3

0.32

0.14

2

x_3

6 8 1

0.14

0.06

0

x_4

3 1 3

0.29

0.125

0

x_5

3 2 7

0.16

0.07

1



Artificial Bee Colony (ABC)

Ejemplo Onlooker Bee Phase (muestreo aceptación-rechazo cíclico):

$$f = x+y+z$$



Onlooker Bees

	Fuente de Alimento	Prob	Fit	Trial
x_1	6 9 8	0.09	0.043	0
x_2	1 2 3	0.32	0.14	2
x_3	6 8 1	0.14	0.06	0
x_4	3 1 3	0.29	0.125	0
x_5	3 2 7	0.16	0.07	1

$r=0.70$

Artificial Bee Colony (ABC)

Ejemplo Onlooker Bee Phase (muestreo aceptación-rechazo cíclico):

$$f = x+y+z$$

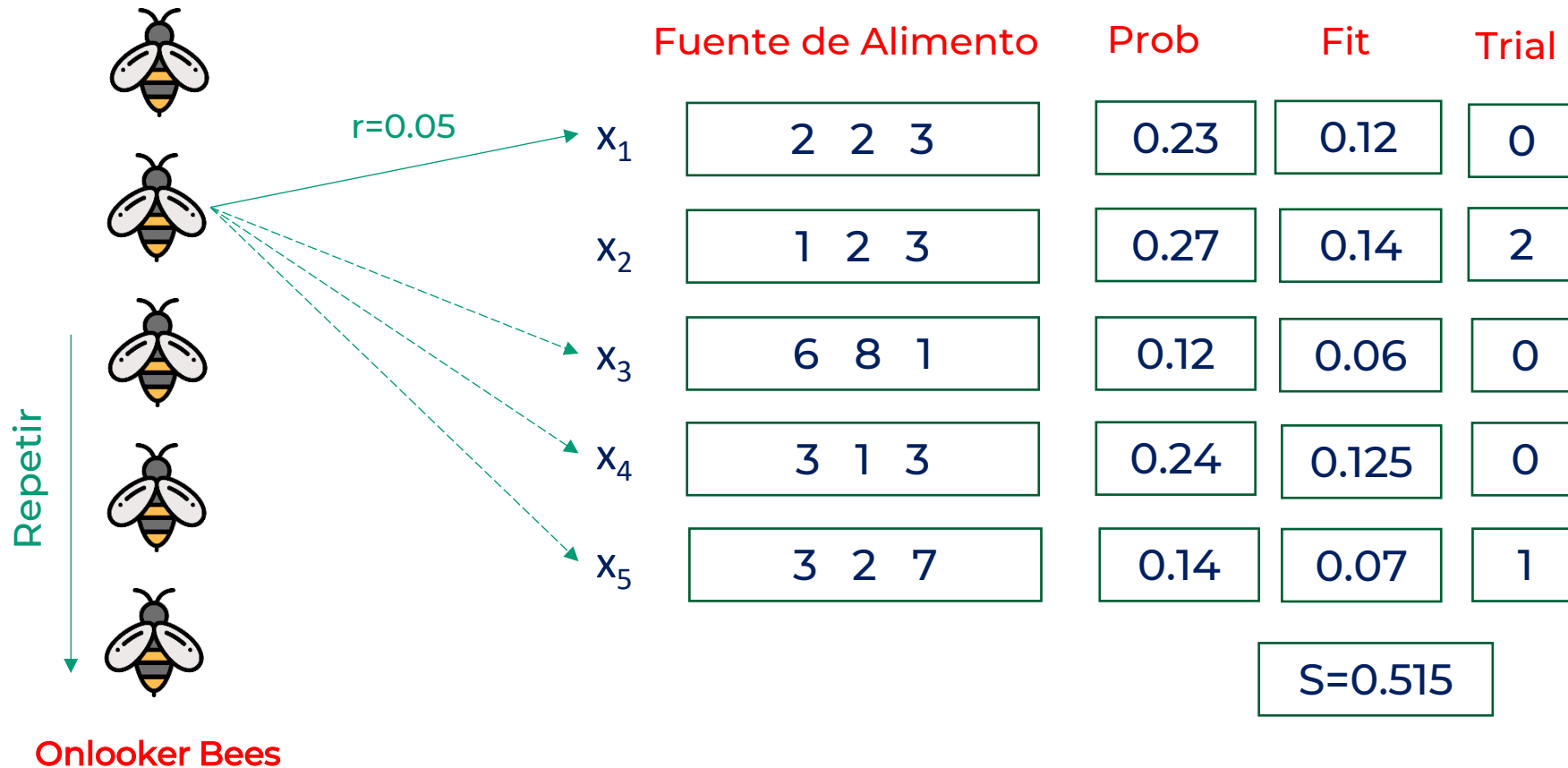


Onlooker Bees

Artificial Bee Colony (ABC)

Ejemplo Onlooker Bee Phase (muestreo aceptación-rechazo cíclico):

$$f = x + y + z$$



Artificial Bee Colony (ABC)

Algoritmo Básico (continuación):

3. Fase de abejas exploradoras (scout)

- Después de que todas las abejas empleadas y observadoras completan sus búsquedas, se verifica si hay alguna fuente que no ha sido mejorada a través de un número predeterminado de ciclos, para abandonarla. Las **abejas exploradoras eligen una fuente nueva** para sustituirla.
- Para lo anterior, aparece el concepto de **límite**, un entero definido por el usuario. El valor del límite se suele calcular usando: **limite = $N \times D$** . Donde $N = SN/2$; D =cantidad de variables.
- Fuentes de alimento cuyo $trials > limite$ potencialmente pueden ser sustituidas.



Artificial Bee Colony (ABC)

Algoritmo Básico (continuación):

3. Fase de abejas exploradoras (scout)

- Las abejas exploradoras ingresan para descubrir accidentalmente fuentes de alimentos desconocidas. Esta operación puede definirse de la siguiente manera:

$$x_i = x_{min} + rand(0,1) * (x_{max} - x_{min}) \quad (4)$$

Este proceso ayuda a evitar soluciones sub-óptimas.

- Este nuevo valor reemplaza al anterior **SIN selección voraz** (greedy).



Artificial Bee Colony (ABC)

Algoritmo Básico (continuación):

3. Fase de abejas exploradoras (scout)

- Si existe por lo menos una fuente de alimento con $\text{trials} > \text{limite}$, se entra en esta fase.
- La implementación clásica (Karaboga, 2005) abandona solo 1 fuente de alimento. Si varios valores poseen $\text{trials} > \text{limite}$, la fuente de alimento a reemplazar se escoge el que tiene mayor cantidad de trials. Si hay más de 1 solución con una misma cantidad de trials, se escoge al azar.
- Hay varias maneras de implementar la selección. Algunos abandonan todas las fuentes con $\text{trials} > \text{limite}$ para aumentar la diversidad (multi-scout)
- Esta etapa puede eliminar la mejor solución => **guardar la mejor solución encontrada antes de empezar esta fase.**

Artificial Bee Colony (ABC)

Ejemplo Scout Bee Phase (limite=1):

Resultado de la fase Onlooker Bee

Fuente de Alimento	Fit	Trial
2 2 3	0.12	0
1 2 3	0.14	2
6 8 1	0.06	0
3 1 3	0.125	0
3 2 7	0.07	1



2 8 5

0.06

0

Artificial Bee Colony (ABC)

Ejemplo Scout Bee Phase (limite=1):

Fuente de Alimento	Fit	Trial
2 2 3	0.12	0
2 8 5	0.06	0
6 8 1	0.06	0
3 1 3	0.125	0
3 2 7	0.07	1

Notar que el reemplazo se hace SIN importar el valor del fit; es decir, NO es greedy.

Artificial Bee Colony (ABC)

- (1) Generate the initial population x_i ($i=1,2,\dots,SN$), using (1)
- (2) Evaluate the fitness($\text{fit}(x_i)$) of the population
- (3) $g = 0$
- (4) Repeat
- (5) For each **employed bee**{
 Produce new solutions $v_{i,g}$ in the neighbourhood of x_i , using (2)
 Calculate its fitness value $\text{fit}(v_{i,g})$
 Apply the selection process between x_i and $v_{i,g}$
 For each solution (x_i){ Calculate the probability values p_i , using (3) }
- (6) For each **onlooker bee**{
 Select a solution x_i depending on p_i
 Produce new solutions $v_{i,g}$ from the solutions x_i , using (2)
 Calculate its fitness value $\text{fit}(v_{i,g})$
 Apply the selection process}
- (7) If there is an abandoned solution x_i then
 replace it with a new randomly produced solution x_i by the **scout bee**, using (4)
 Memorize the best solution achieved so far
- (8) $g = g + 1$
- (9) Until $g = \text{max_iter}$

Artificial Bee Colony (ABC)

Parámetros:

Los parámetros del algoritmo son los siguientes:

1. **S**: swarm number. Tamaño del enjambre. Seleccionado por usuario.
2. **SN**: es el número de fuentes de alimento. Normalmente, $SN=S/2$.
3. **max_iter**: el número total de iteraciones que ejecutará ABC
4. **límit**: número de ciclos que será conservada una solución (fuente de alimento) puede permanecer sin mejorar antes de ser reemplazada por una nueva solución generada por una abeja exploradora.



Artificial Bee Colony (ABC)

Consideraciones:

- **Abejas empleadas:** su número es proporcional al número de fuentes de alimento, y su función es evaluar y modificar las soluciones actuales para mejorarlas. Si la nueva posición no es mejor entonces se mantiene la posición actual.

$$\# \text{ Abejas empleadas} = \# \text{ fuentes de alimento} = SN$$

- **Abejas observadoras:** su número es proporcional al número de fuentes de alimento. Estas abejas escogerán una fuente de alimento, con base en la información que comparten las abejas empleadas mediante la danza. Esta danza se puede simular con método de la ruleta o torneo de tamaño "t", donde la fuente de alimento con mejor valor de la función objeto es seleccionada.

$$\# \text{ Abejas observadoras} = \# \text{ Abejas empleadas} = \# \text{ fuentes de alimento} = SN$$



Artificial Bee Colony (ABC)

Consideraciones:

- ABC es un algoritmo de naturaleza estocástica que combina la búsqueda local (realizada por las abejas empleadas y observadoras), y también la búsqueda global (por las abejas exploradoras) para equilibrar el proceso de **exploración** (nuevo) y **explotación** (refinamiento).
- Las principales ventajas del algoritmo ABC sobre otros métodos de optimización son:
 - Es simple de implementar.
 - Tiene pocos parámetros de control.
 - Es robusto y altamente flexible
 - Fácil de combinar con otros métodos
 - Rápida convergencia al combinar procesos de exploración y explotación.

Artificial Bee Colony (ABC)

Consideraciones:

- ABC tiene algunas debilidades cuando se pone en práctica.
 - Este método requiere evaluar el fitness en cada nueva fase de abejas del algoritmo para mejorar su rendimiento. Necesita una gran cantidad de evaluaciones de la función objetivo.
 - Se ralentiza cuando la población de soluciones aumenta el costo computacional.
 - Tiene muchas iteraciones y, por lo tanto, requiere una gran capacidad de memoria.

Bibliografía



- E. Bonabeau, et. al , 1999. Swarm Intelligence. From Nature to Artificial Systems. Oxford University Press.
- Eberhart, R. C. and Kennedy, J. A new optimizer using particle swarm theory. Proceedings of the Sixth International Symposium on Micromachine and Human Science, Nagoya, Japan. pp. 39-43,1995
- Kennedy, J. and Eberhart, R. C. Particle swarm optimization. Proceedings of IEEE International Conference on Neural Networks, Piscataway, NJ. pp. 1942-1948, 1995.
- A. Moraglio, C. Di Chio, and R. Poli. Geometric Particle Swarm Optimization. In 10th European conference on Genetic Programming (EuroGP2007), volume 4445 of Lecture Notes in Computer Science. Springer, Abril 2007.

Bibliografía



- D. Karaboga. An idea based on honey bee swarm for numerical optimization. Tr06, Erciyes University, Engineering Faculty, Computer Engineering Department, 2005.
- Raúl Benítez, Gerard Escudero, Samir Kanaan. Inteligencia artificial avanzada.
- M. Dorigo, Optimization, Learning and Natural Algorithms, PhD thesis, Politecnico di Milano, Italy, 1992.
- Marco Dorigo, Mauro Birattari, and Thomas Stutzle, Universite Libre de Bruxelles, BELGIUM.
- R. Storn, 1997. Differential Evolution, A simple and efficient heuristic of strategy for global optimization over continuous spaces. Journal of Global Optimization, 11 (1997) 341-359.

¡Gracias!

